
FastAPI First Endpoints Lab (Practical)

Welcome to the practical lab for FastAPI! In this session, you will write your first backend server, configure Pydantic data validation, and test your endpoints using both **Postman** and FastAPI's built-in Swagger documentation.

1. Concept Overview

In this lab, you will:

- Set up an project folder.
 - Define **Pydantic models** to strictly validate incoming JSON payloads (such as ML feature inputs).
 - Test your API endpoints using **Postman**, a popular tool for designing, building, and testing APIs.
-

2. Step-by-Step Lab Instructions

Step 1: Create Workspace Directory

Open your terminal (or VS Code integrated terminal) and run the following commands to create a workspace directory:

```
# Create project folder and navigate into it
mkdir fullstack-ml-backend
cd fullstack-ml-backend
```

Step 2: Create `main.py` and Define Endpoints

Create a new file named `main.py` in your project folder and add the following code:

```
# main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

# Initialize FastAPI app
```

```

app = FastAPI(title="ML Prediction Backend", version="1.0")

# Configure CORS so your React frontend (port 5173) can communicate with
this backend
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Define a Pydantic model for incoming prediction request payloads
class PredictionInput(BaseModel):
    feature1: float
    feature2: float

# Root GET endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to the FastAPI Backend for ML
Predictions!"}

# POST prediction endpoint with data validation
@app.post("/predict")
def run_prediction(data: PredictionInput):
    # Simulated model inference logic (will be replaced with a .pkl model
    later)
    calculated_prediction = (data.feature1 + data.feature2) * 1.5

    return {
        "status": "success",
        "input_received": {
            "feature1": data.feature1,
            "feature2": data.feature2
        },
        "prediction": round(calculated_prediction, 2)
    }

```

Step 4: Run the Development Server

Start your FastAPI server using **Uvicorn** with the `--reload` flag (which automatically restarts the server when you save code changes):

```
uvicorn main:app --reload
```

You should see output similar to:

```

INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [12345] using StatReload
INFO:      Started server process [67890]
INFO:      Waiting for connection.

```

3. Testing Your API with Postman & Swagger UI

Method A: Testing with Postman

1. **Download & Open Postman:** If you haven't already, download Postman from the [Postman Website](#).

2. **Create a New Request:** Click the **+** button to open a new request tab.

3. **Configure Request Type & URL:**

- Change the request method dropdown to **POST**.
- Enter the URL: `[http://127.0.0.1:8000/predict]`
(`http://127.0.0.1:8000/predict`)

4. **Add JSON Payload:**

- Go to the **Body** tab below the URL bar.
- Select **raw** and change the format dropdown from Text to **JSON**.
- Paste the following test JSON body:

```
{  
  "feature1": 1500,  
  "feature2": 3  
}
```

5. **Send Request:** Click the blue **Send** button. You should receive a **200 OK** status response containing your calculated prediction in the lower response panel!

Method B: Testing with Built-In Swagger UI

FastAPI automatically generates interactive documentation. Open your browser and navigate to:

- `[http://127.0.0.1:8000/docs](http://127.0.0.1:8000/docs)`
- Click on the `POST /predict` route, click **Try it out**, enter your numeric feature values, and click **Execute** to test your API directly in the browser.

4. Reference Links & Further Reading

- **FastAPI First Steps Tutorial:** Official walkthrough on creating paths and parameters.
 - **Postman Learning Center:** Comprehensive guide on testing REST APIs.
-